# Bean Scope-ları və Dependency Injection

Spring Praktika · README (tapşırıq nəticələri, şərhlər və ekran görüntüləri)
Layihə: userAPI · UserInfoController → UserInfoService → UserRepository · 13.10.2026

| Bölmə | Mövzu | Əsas nəticə |
|---|---|---|
| 1 | Singleton | Bean 1 dəfə yaranır, bütün thread-lər paylaşır; @PostConstruct / @PreDestroy işləyir |
| 2 | Prototype | Birbaşa inject → 1 auditId; ObjectProvider → hər çağırışda yeni auditId; destroy çağırılmır |
| 3 | Request | Controller və Service eyni requestId; proxyMode olmadan tətbiq açılmır; CGLIB proxy |
| 4 | Session | JSESSIONID cookie-yə görə hər brauzerin öz siyahısı; session bitəndə @PreDestroy |
| 5 | Dependency Injection | NoUniqueBeanDefinitionException → @Primary / @Qualifier; test constructor injection ilə |

## Bölmə 1 · Singleton

### Tapşırıq 3 · @PostConstruct

> **Şərh.** Tətbiq açılanda `@PostConstruct` işləyir və konsola **UserRepository hazirdir, 3 user yuklendi** yazılır. Eyni logda UserRepository, UserInfoService və UserInfoController konstruktorlarının hər birinin yalnız **bir dəfə** çağırıldığı görünür (hashCode-lar: 162393366, 1636947672, 671986597).




*Tətbiq açılışı: ... yaradildi log-ları və 'hazirdir, 3 user yuklendi'*

## Tapşırıq 4 · @PreDestroy

**Şərh.** Tətbiq dayandırılanda (SIGINT) `@PreDestroy` işləyir: **UserRepository baglanir, storage temizlenir (4 user)**. Graceful shutdown bitdikdən sonra storage təmizlənir.




*Shutdown log-u: storage təmizlənir (4 user)*

## Tapşırıq 5 · NoSuchElementException

**Şərh.** Mövcud olmayan id (999) ilə sorğuda `java.util.NoSuchElementException: User tapilmadi: id=999` atılır. Stack trace zənciri Repository → Service → Controller ardıcıllığını göstərir.



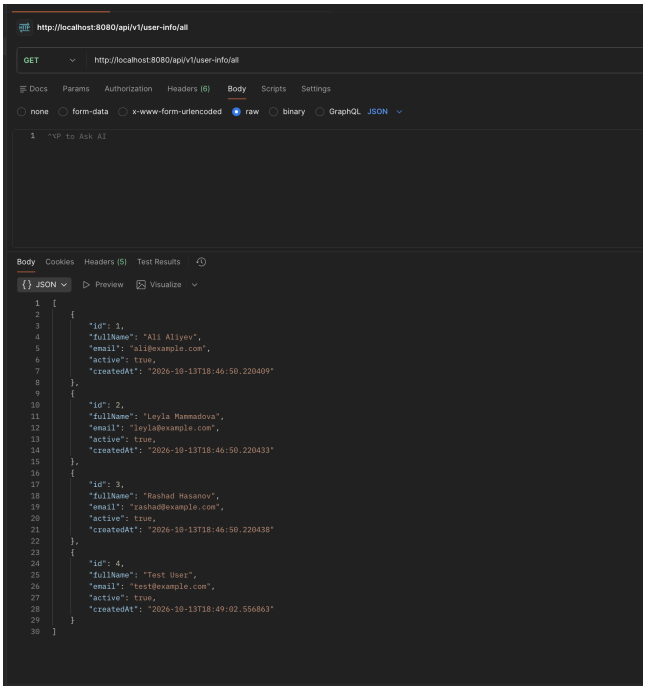

*getUserById(999) xətası*

## Tapşırıq 6 · Konstruktor log-u neçə dəfə çıxdı?

**Şərh. Cavab: yalnız 1 dəfə** — tətbiq açılanda. Controller, Service və Repository **singleton** bean-dir (Spring-də default scope): container hər birindən bir instance yaradır və bütün sorğularda eyni obyekt istifadə olunur. Ona görə 10 sorğu göndərəndə də konstruktor log-u təkrar çıxmır, hashCode həmişə eynidir.

### Tapşırıq 7 · POST /create və GET /all

> **Şərh.** Yeni user yaradıldıqdan sonra **GET /all 4 user** qaytarır: 3 başlanğıc user (Ali, Leyla, Rashad) + yeni yaradılan Test User (id=4). ID-lər `AtomicLong.incrementAndGet()` ilə ardıcıl verilir.




*Postman: GET /api/v1/user-info/all → 4 user*

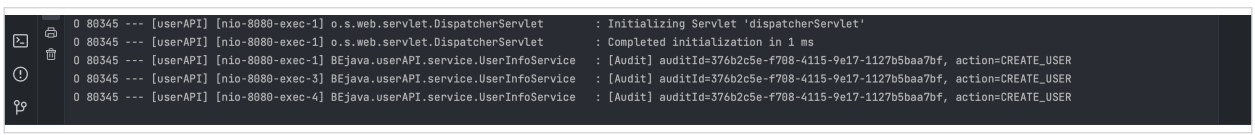
### Tapşırıq 8 · HashMap və long counter niyə təhlükəlidir?

> **Şərh.** Singleton bean **bir instance-dır, amma onu eyni anda çox thread istifadə edir** — Tomcat hər sorğunu ayrı worker thread-də icra edir (loglarda nio-8080-exec-1, exec-3, exec-4 görünür; default olaraq 200-ə qədər thread). **HashMap** thread-safe deyil: paralel `put` zamanı (xüsusilə resize olanda) yazılar itə, struktur korlana bilər. **long counter**-də `counter++` atomik deyil (oxu → artır → yaz), iki thread eyni dəyəri oxuyub **eyni ID** verə bilər. Həll: `ConcurrentHashMap` və `AtomicLong`.

## Bölmə 2 · Prototype

### Tapşırıq 11 · Səhv variant — birbaşa inject

> **Şərh.** UserAuditLog service konstruktoruna birbaşa inject edildikdə 3 sorğuda **3 dəfə eyni auditId** (376b2c5e-...) görünür — **fərqli auditId sayı: 1**. Səbəb: singleton Service yaranarkən prototype bean **yalnız bir dəfə** inject olunur və sonra həmin obyekt saxlanılır.




*3 sorğu — eyni auditId*

### Tapşırıq 12 · Düzgün variant — ObjectProvider

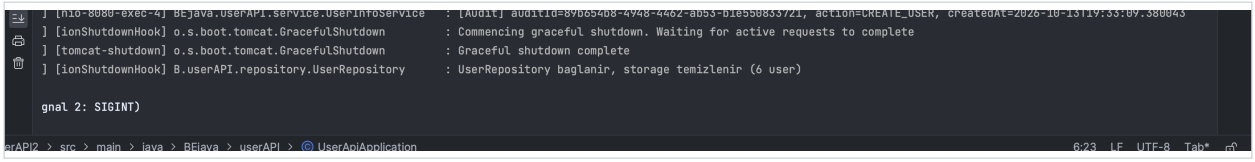
**Şərh.** `ObjectProvider<UserAuditLog>.getObject()` hər createUser çağırışında **yeni obyekt** verir: 3 sorğuda 3 fərqli auditId (da64bdfd..., e467ec99..., 89b654b8...), hər biri üçün 'UserAuditLog yaradildi' log-u çıxır.




*3 sorğu — 3 fərqli auditId, action=CREATE_USER*

### Tapşırıq 13 · Prototype və @PreDestroy

**Şərh.** Tətbiq bağlananda **UserAuditLog üçün @PreDestroy log-u çıxmadı** (shutdown logunda yalnız UserRepository-nin 'storage temizlenir (6 user)' sətri var). Səbəb: Spring prototype bean-i yaradıb təhvil verir və **onun həyat dövrünü sonra izləmir** — destroy callback-ləri prototype bean-lər üçün çağırılmır. Təmizləməni kod özü etməlidir.

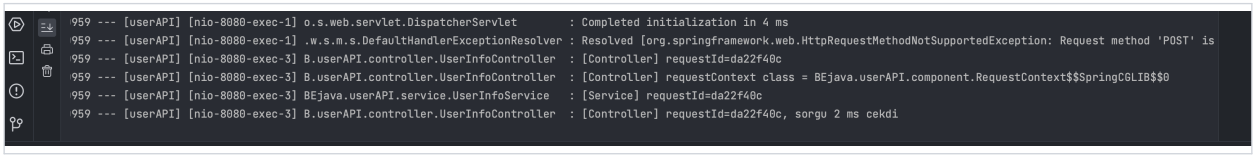



*Shutdown: UserAuditLog destroy log-u yoxdur*

# Bölmə 3 · Request

### Tapşırıq 16–17 · requestId və sorğu müddəti

**Şərh.** Eyni sorğuda **Controller və Service eyni requestId** (da22f40c) yazır; Controller sorğu bitəndə müddəti də yazır: **sorgu 2 ms cekdi**. Növbəti sorğuda yeni RequestContext yaranır və requestId dəyişir.




*[Controller] / [Service] requestId=da22f40c, sorgu 2 ms cekdi*

### Tapşırıq 18 · @Scope("request") — proxyMode olmadan

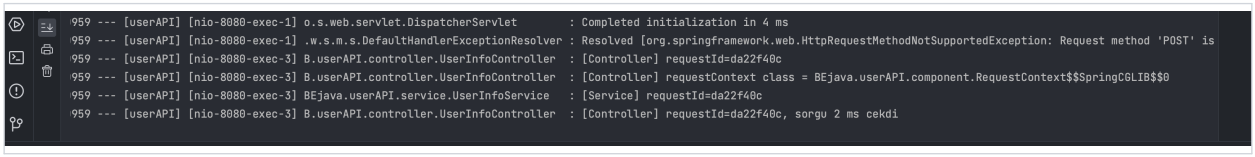
**Şərh.** Tətbiq açılmadı: **Scope 'request' is not active for the current thread; consider defining a scoped proxy for this bean if you intend to refer to it from a singleton**. Səbəb: singleton Controller/Service tətbiq *açılarkən* yaranır, bu anda heç bir HTTP sorğusu yoxdur, request scope isə aktiv deyil. Düzəliş: `@Scope(value = "request", proxyMode = ScopedProxyMode.TARGET_CLASS)` (və ya `@RequestScope`, o proxy-ni özü qurur).




*Xəta: Scope 'request' is not active for the current thread*

### Tapşırıq 19 · requestContext.getClass()

> **Şərh.** Çıxan nəticə: `RequestContext$$SpringCGLIB$$0` — yəni inject olunan obyekt real RequestContext deyil, **CGLIB proxy**-dir. Singleton-a request bean belə inject olunur: singleton-a **proxy** verilir; proxy hər metod çağırışında cari HTTP sorğusuna bağlı **real RequestContext** instance-ını tapıb çağırışı ona yönləndirir. Beləliklə singleton bir dəfə yaranır, amma hər sorğuda fərqli request bean ilə işləyir.

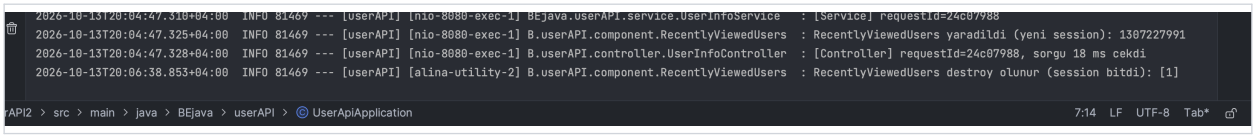



*requestContext class = ...RequestContext$$SpringCGLIB$$0*

# Bölmə 4 · Session

### Tapşırıq 23 · Session timeout = 1m və @PreDestroy

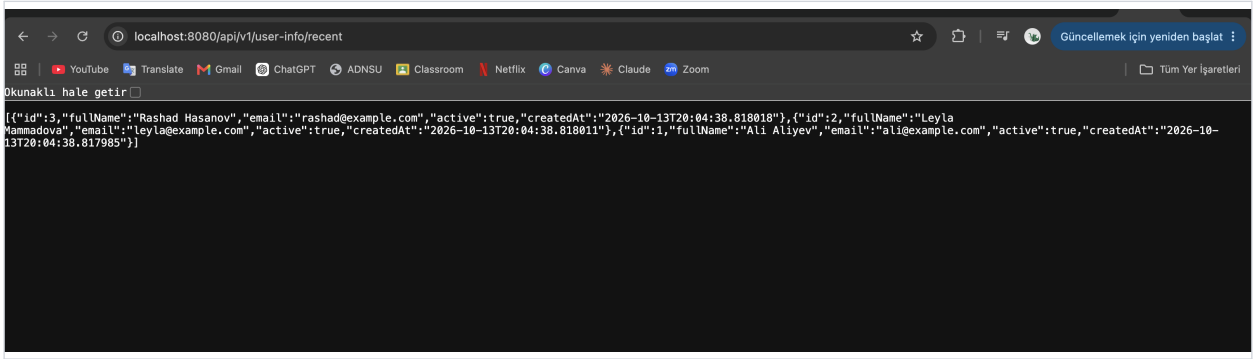
> **Şərh.** Session bitəndə log çıxdı: **RecentlyViewedUsers destroy olunur (session bitdi): [1]**. Session 20:04:47-də yaranıb, destroy 20:06:38-də çıxıb. Timeout 1 dəqiqə olsa da Tomcat bitmiş session-ları arxa fon prosesi ilə (təxminən dəqiqədə bir) yoxlayır, ona görə bağlanma dəqiq 60-cı saniyədə yox, bir qədər sonra baş verir.



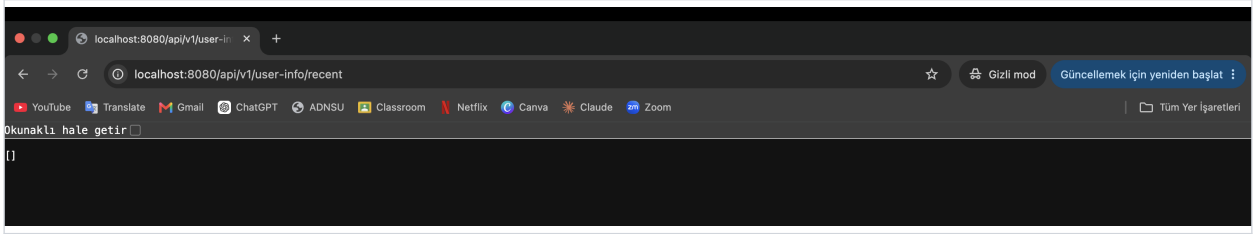

*Yeni session yaradıldı → timeout sonrası destroy log-u*

### Tapşırıq 24 · Chrome, incognito və Postman

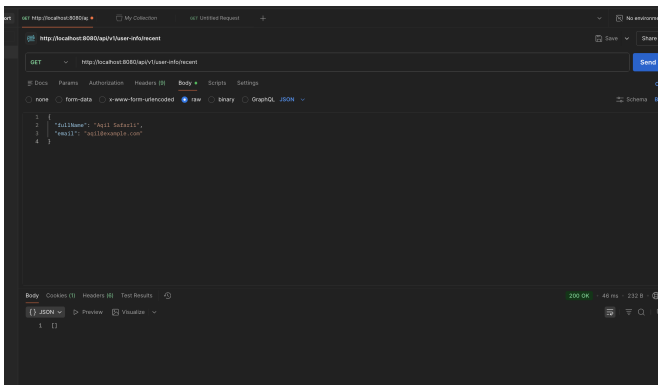
> **Şərh. Chrome**: /recent tam user məlumatı qaytarır — [3, 2, 1] siyahısı təkrarsız və ən yenisi əvvəldə. **Incognito**: **[]** — incognito ayrı cookie anbarı olduğu üçün **yeni JSESSIONID** alır, yeni session = yeni, boş RecentlyViewedUsers. **Postman** da brauzerin cookie-sini bilmir; öz cookie jar-ı var (Cookies (1)) və öz session-u üçün boş siyahı qaytarır. DevTools → Application → Cookies bölməsində **JSESSIONID** cookie-si görünür: server session-u məhz bu cookie ilə tanıyır.



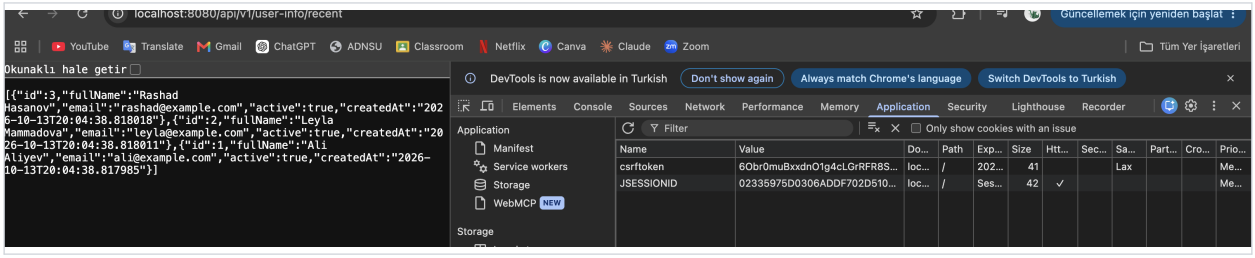

*Chrome: /recent*




*Incognito: /recent → []*



*Postman: /recent → [] (Cookies (1))*




*DevTools: JSESSIONID cookie*

### Tapşırıq 25 · Siyahı request scope-da saxlansaydı?

> **Şərh.** Request scope-da bean **hər sorğuda yenidən yaranır** və sorğu bitəndə atılır. Buna görə /1, /2, /3 çağırışlarında əlavə olunan ID-lər növbəti sorğuya **keçməzdi** — /recent həmişə boş siyahı qaytarardı. İstifadəçiyə aid məlumat sorğular arasında saxlanmalıdırsa session scope lazımdır.

## Bölmə 5 · Dependency Injection

### Tapşırıq 27 · NoUniqueBeanDefinitionException

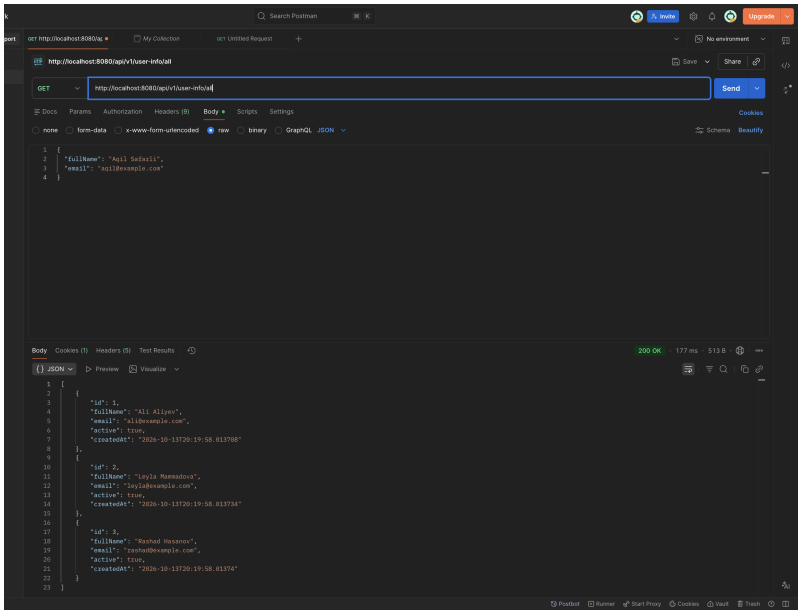
> **Şərh.** İki implementasiya (InMemoryUserRepository və MockUserRepository) olduqda tətbiq açılmır: **Parameter 0 of constructor in UserInfoService required a single bean, but 2 were found**. Spring hansını seçəcəyini bilmir.




*APPLICATION FAILED TO START — 2 bean tapıldı*

## Tapşırıq 28 · Həll 1 — @Primary

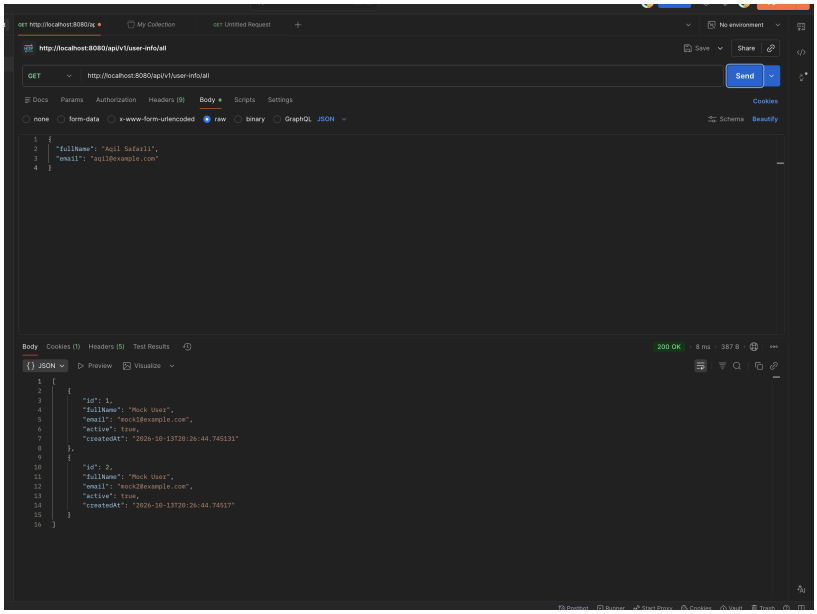
**Şərh.** InMemoryUserRepository-yə **@Primary** verildikdə o default seçilir: **GET /all** yenidən real in-memory user-ləri qaytarır (Ali, Leyla, Rashad).




*@Primary: /all → in-memory user-lər*

## Tapşırıq 29 · Həll 2 — @Qualifier

**Şərh.** `@Qualifier("mockUserRepository")` ilə service mock repository-ni seçir: **GET /all** adı **Mock User** olan 2 user qaytarır.




*@Qualifier: /all → 2 Mock User*

## Tapşırıq 31 · ConsoleNotificationService

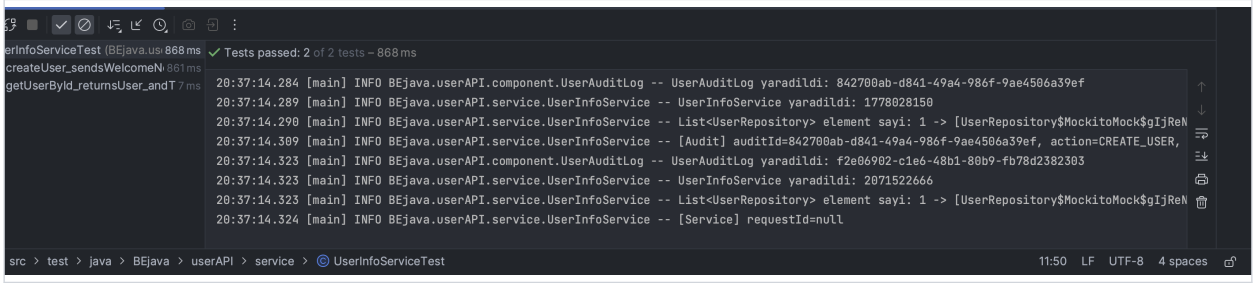
> **Şərh.** createUser çağırılanda audit log-dan sonra **Xos geldin, aqil@example.com!** log-u çıxır — optional inject olunan NotificationService implementasiyası işə düşür.




*createUser → 'Xos geldin' log-u*

## Tapşırıq 32–33 · Spring-siz unit test, List<UserRepository>

> **Şərh.** Mockito ilə yazılmış test **Spring context olmadan** işləyir: 2 testin hər ikisi keçdi (**Tests passed: 2 of 2**). Bu, `new UserInfoService(mock)` ilə **constructor injection** sayəsində mümkündür. **Field injection**-da asılılıq private field-dir və yalnız Spring reflection ilə doldurur; Spring olmadan field `null` qalır (NullPointerException) və ona mock vermək üçün konstruktor/setter yoxdur. Buna görə constructor injection-a qayıdırıq. Testdə konstruktora verilən List<UserRepository> **1 element** saxlayır (mock). Real tətbiqdə context-də iki UserRepository bean olduğuna görə list 2 element almalıdır.




*Tests passed: 2 of 2; List<UserRepository> element sayi: 1*